

Homework 06

Question 1: Compare KNN and logistic regression

Use `breast-cancer.csv`, which contains 569 cases from the [Wisconsin Diagnostic Breast Cancer dataset](#). Predict y (1 for malignant and 0 for benign) using `mean_radius`, `mean_texture`, and `mean_smoothness`.

Set a random seed and randomly split the observations into 70% training and 30% test data. Use the same split for every model. Standardize the predictors using the training means and standard deviations, and apply the same transformations to the test data.

Fit ordinary logistic regression with an intercept and no penalty, and KNN with Euclidean distance, equal neighbor weights, and

$$k \in \{5, 15, 25, 35, 45\}.$$

Using the estimated probabilities for $y = 1$ on the test data, plot all six ROC curves in one figure and report their AUC values. Which of the six fitted models do you prefer? Give a brief justification based on your results.

```
library(class)
library(pROC)
library(ggplot2)
```

Warning: package 'ggplot2' was built under R version 4.4.3

```
dat <- read.csv("data/breast-cancer.csv")
dat <- dat[, c("y", "mean_radius", "mean_texture", "mean_smoothness")]
str(dat)
```

```
'data.frame': 569 obs. of 4 variables:
 $ y          : int  1 1 1 1 1 1 1 1 1 1 ...
 $ mean_radius : num  18 20.6 19.7 11.4 20.3 ...
 $ mean_texture : num  10.4 17.8 21.2 20.4 14.3 ...
 $ mean_smoothness: num  0.1184 0.0847 0.1096 0.1425 0.1003 ...
```

Split and standardize

```
set.seed(123)
n <- nrow(dat)
train_idx <- sample(seq_len(n), size = round(0.7 * n))

train <- dat[train_idx, ]
test <- dat[-train_idx, ]

preds <- c("mean_radius", "mean_texture", "mean_smoothness")

# standardize using training means and sds only
mu <- colMeans(train[, preds])
sdv <- apply(train[, preds], 2, sd)
```

```

train_s <- train
test_s <- test
train_s[, preds] <- sweep(sweep(train[, preds], 2, mu, "-"), 2, sdv, "/")
test_s[, preds] <- sweep(sweep(test[, preds], 2, mu, "-"), 2, sdv, "/")

```

Fit models

```

# logistic regression, no penalty
fit_glm <- glm(y ~ mean_radius + mean_texture + mean_smoothness,
              data = train_s, family = binomial)
p_glm <- predict(fit_glm, newdata = test_s, type = "response")

# KNN: estimated P(y = 1) is the fraction of the k neighbors with y = 1
ks <- c(5, 15, 25, 35, 45)
knn_prob <- function(k) {
  fit <- knn(train = train_s[, preds], test = test_s[, preds],
            cl = factor(train_s$y, levels = c(0, 1)),
            k = k, prob = TRUE)
  win_prob <- attr(fit, "prob") # proportion of votes for the winning class
  ifelse(fit == "1", win_prob, 1 - win_prob)
}

p_knn <- lapply(ks, knn_prob)
names(p_knn) <- paste0("KNN (k=", ks, ")")

all_probs <- c(list("Logistic" = p_glm), p_knn)

```

ROC curves and AUC

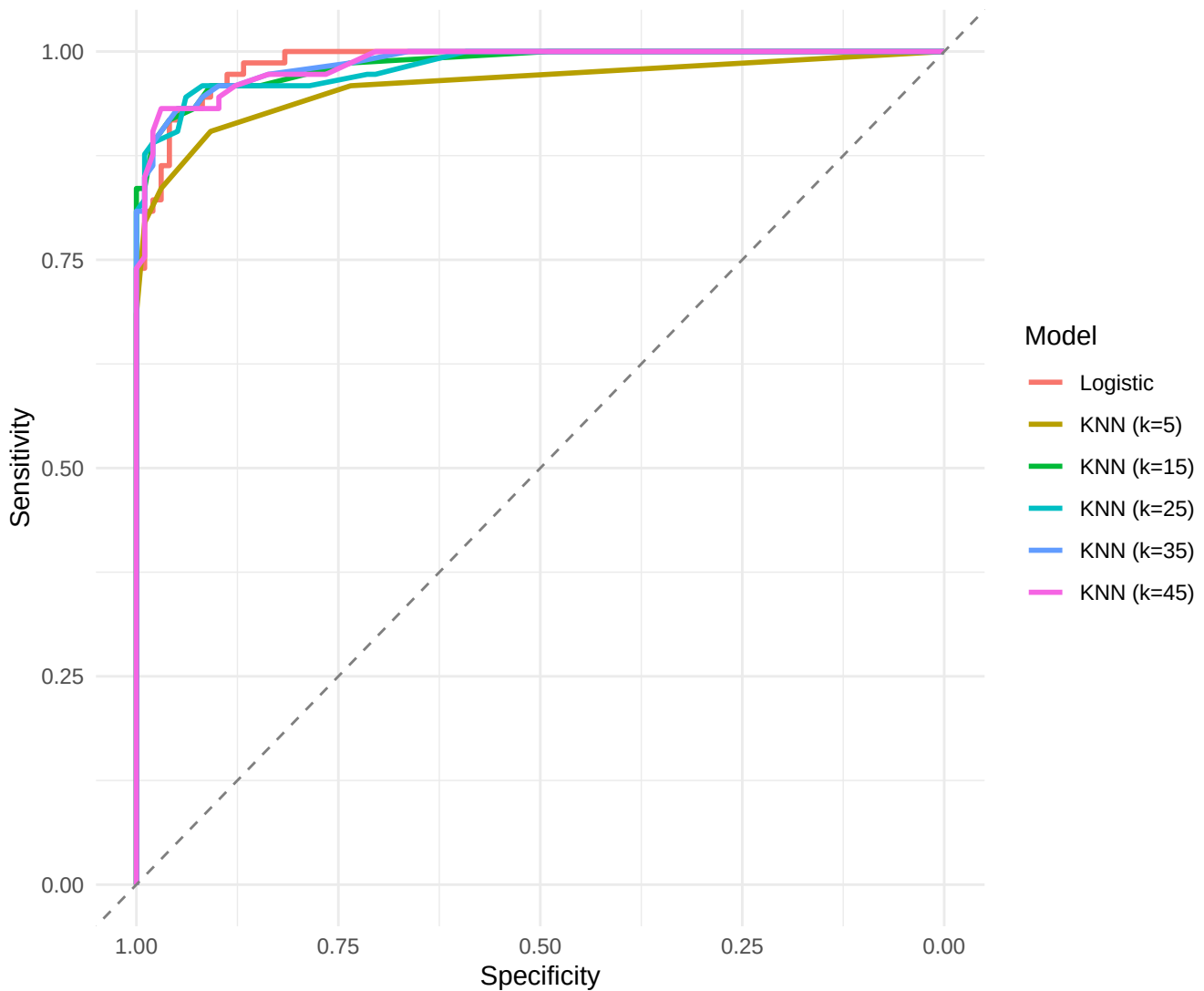
```

rocs <- lapply(all_probs, function(p) {
  roc(response = test_s$y, predictor = p, levels = c(0, 1),
      direction = "<", quiet = TRUE)
})

ggroc(rocs, linewidth = 1) +
  geom_abline(slope = 1, intercept = 1, linetype = "dashed", color = "grey50") +
  labs(title = "ROC curves on test data", x = "Specificity", y = "Sensitivity",
       color = "Model") +
  theme_minimal()

```

ROC curves on test data



```
auc_tab <- data.frame(Model = names(rocs),
                      AUC = round(sapply(rocs, function(r) as.numeric(auc(r))), 4))
rownames(auc_tab) <- NULL
auc_tab
```

	Model	AUC
1	Logistic	0.9866
2	KNN (k=5)	0.9586
3	KNN (k=15)	0.9843
4	KNN (k=25)	0.9823
5	KNN (k=35)	0.9860
6	KNN (k=45)	0.9851

I prefer ordinary logistic regression. It has the highest test AUC (0.9866), though KNN with $k = 35$ (0.9860), $k = 45$ (0.9851), and $k = 15$ (0.9843) are essentially tied with it. KNN with $k = 5$ is clearly the weakest (0.9586): its ROC curve sits visibly below the others, because with only 5 neighbors the estimated probabilities take just six distinct values (0, 0.2, ..., 1) and are noisy. Since the test set has only about 171 cases, differences in the third decimal place are well within sampling noise, so the AUC alone can't separate logistic regression from the larger- k KNN fits. Given that tie, I choose logistic regression because it is the simplest model, has no tuning parameter, produces smooth probability estimates, and is cheap to evaluate on new data.

Question 2: Training and test ROC curves

Use the same data, training/test split, and standardization as in Question 1. Fit KNN with Euclidean distance, equal neighbor weights, and

$$k \in \{1, 3, 5, 10, 20\}.$$

Use each fitted model to estimate probabilities for both the training and test observations. For training predictions, include each observation itself among its k nearest neighbors.

For each k , use a separate panel to show the training and test ROC curves together. Report both AUC values.

Explain the difference between training and test performance and how it changes with k . In particular, explain what happens at $k = 1$ and why training performance may not reflect performance on new observations.

```
ks2 <- c(1, 3, 5, 10, 20)

# P(y = 1) estimated as the fraction of the k nearest neighbors with y = 1
knn_p <- function(train_x, test_x, cl, k) {
  fit <- knn(train = train_x, test = test_x,
            cl = factor(cl, levels = c(0, 1)), k = k, prob = TRUE)
  win_prob <- attr(fit, "prob")
  ifelse(fit == "1", win_prob, 1 - win_prob)
}

roc_df <- data.frame()
auc_rows <- data.frame()

for (k in ks2) {
  # test predictions
  p_te <- knn_p(train_s[, preds], test_s[, preds], train_s$y, k)
  # training predictions: the training set is its own "test" set,
  # so each observation is one of its own k nearest neighbors
  p_tr <- knn_p(train_s[, preds], train_s[, preds], train_s$y, k)

  r_te <- roc(test_s$y, p_te, levels = c(0, 1), direction = "<", quiet = TRUE)
  r_tr <- roc(train_s$y, p_tr, levels = c(0, 1), direction = "<", quiet = TRUE)

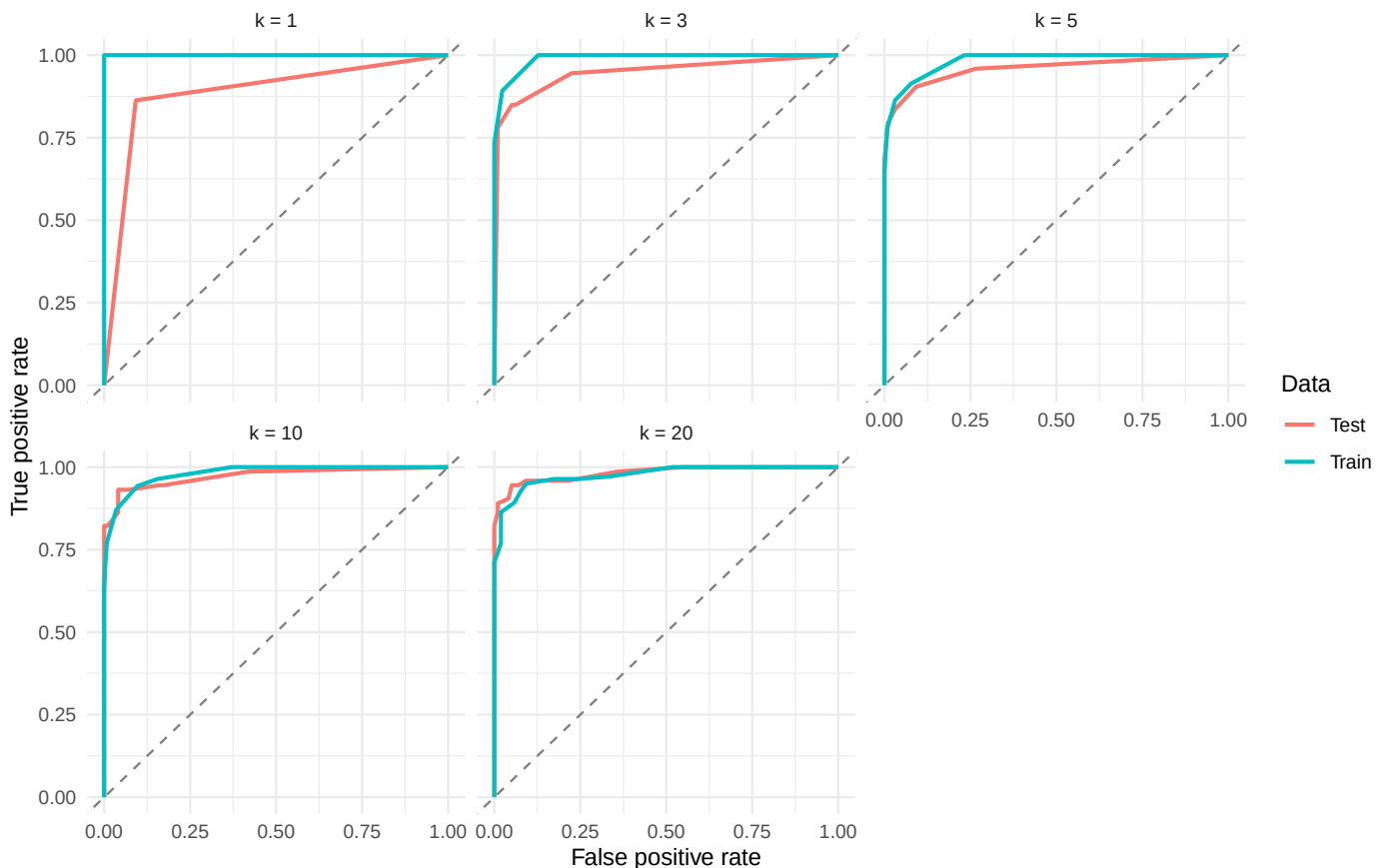
  for (nm in c("Train", "Test")) {
    r <- if (nm == "Train") r_tr else r_te
    cc <- coords(r, "all", ret = c("specificity", "sensitivity"),
                transpose = FALSE)
    roc_df <- rbind(roc_df,
                  data.frame(k = paste0("k = ", k), Data = nm,
                              fpr = 1 - cc$specificity,
                              tpr = cc$sensitivity))
  }

  auc_rows <- rbind(auc_rows,
                    data.frame(k = k,
                                Train_AUC = round(as.numeric(auc(r_tr)), 4),
                                Test_AUC = round(as.numeric(auc(r_te)), 4)))
}

roc_df$k <- factor(roc_df$k, levels = paste0("k = ", ks2))

ggplot(roc_df, aes(x = fpr, y = tpr, color = Data)) +
  geom_path(linewidth = 0.9) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey50") +
  facet_wrap(~ k) +
  labs(title = "Training vs test ROC curves for KNN",
       x = "False positive rate", y = "True positive rate") +
  theme_minimal()
```

Training vs test ROC curves for KNN



auc_rows

	k	Train_AUC	Test_AUC
1	1	1.0000	0.8856
2	3	0.9900	0.9467
3	5	0.9818	0.9586
4	10	0.9805	0.9727
5	20	0.9761	0.9821

At $k = 1$ the training AUC is exactly 1.0000 because each training point is its own nearest neighbor, but the test AUC is only 0.8856, so the model has memorized noise rather than learned a pattern. As k increases, the training AUC falls (to 0.9761 at $k = 20$) and the test AUC rises (to 0.9821), so the two curves nearly meet. Training performance doesn't reflect performance on new observations because the model has already seen those labels, and using it to choose k would wrongly pick $k = 1$, which has the worst test AUC.

Question 3: Peer evaluation of Homework 5

You will receive two Homework 5 submissions to review. The instructor's GitHub account, **rqzhu-aide**, will open a **GitHub Issue** in the repository containing your Homework 5 submission. It will contain links to two other students' repositories, where you can find their submitted homework reports.

Open your repository's **Issues** tab on GitHub to find the assignment. You may ask AI to explain GitHub Issues or help you locate the assignment.

Choose **one specific mistake or item for improvement** from any question in either report.

1. **Identify the mistake or item for improvement.** State which report and question you selected, and briefly describe what is wrong or could be improved. Examples include incorrect code, a calculation error, a misinterpretation of results, a plot that could communicate the findings more clearly, or anything else you think could be done better.

2. **Explain your assessment.** Quote the relevant text, code, or figure caption exactly. Explain why the selected answer is incorrect or how it could be improved, referring to its reasoning, calculations, code, figures, or interpretation. **Complete this part yourself, without AI assistance.**
3. **Provide a correction.** Write a corrected or improved answer to the selected question, including any calculations, code, or figures needed.

After completing your review, **open a GitHub Issue in the repository containing the report you selected.** Identify the report and question, and include your assessment and a detailed revision plan explaining what the author should change and why. Include a link to your posted GitHub Issue in your Homework 6 answer.

GitHub Issue: <https://github.com/amoghu26/stat432-fall2026/issues/2>